

# Parallel and distributed Computing

## Lecture 31

Bazam-e-Jahan

Lecturer

The University of Faisalabad

# Lecture 31

## A Decentralized Algorithm

# Characteristics of Decentralized Algo

- no machine has complete information about the system state.
- Machine make decision based only on local information
- Failure of one machine does not ruin the algorithm
- There is no implicit assumption that a global clock exists

- Based on the distributed hash table DHT system structure previously introduced
  - Peer-to-peer
  - object names are hashed to find the successor node that will store them
- Here, we assume that  $n$  replicas of each object are stored

- Every replica has a coordinator that controls access to it (the coordinator is the node that stores it)
- For a process to use the resource it must receive permission from  $m > n/2$  coordinators
- This guarantees exclusive access as long as a coordinator only grants access to one process at a time

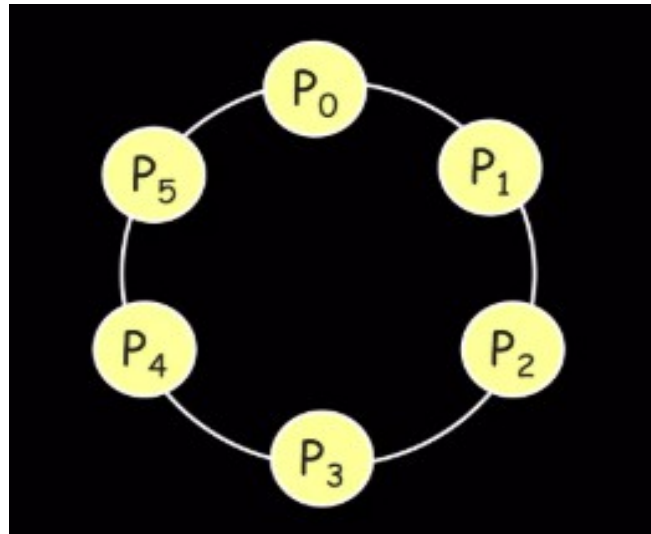
- The coordinator notifies the requester when it has been denied access as well as when it is granted.
  - Requester must "count the votes", and decide whether or not overall permission has been granted or denied
- If a process (requester) gets fewer than  $m$  votes it will wait for a random time and then ask again

# Lecture 32

## A Token Ring Algorithm

# Token Ring algorithm

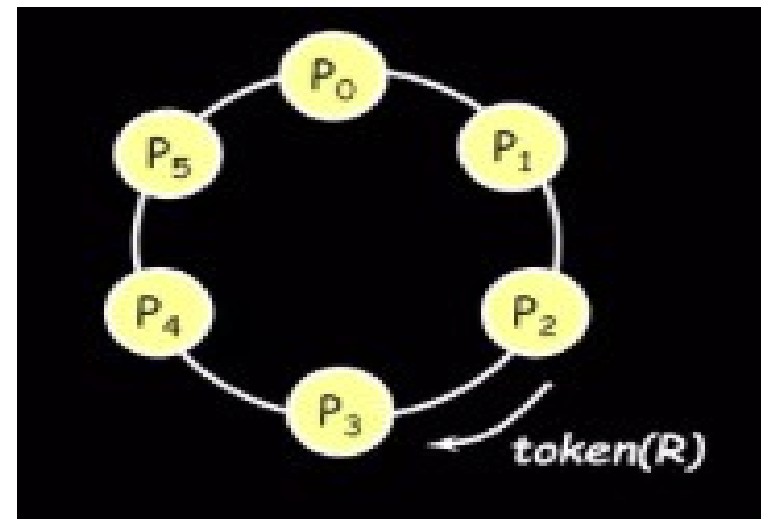
- Assume known group of processes
  - Some ordering can be imposed on group
  - Construct logical ring in software
  - Process communicates with neighbor





# Token Ring algorithm

- Initialization
  - Process 0 gets token for resource R
- Token circulates around ring
  - From  $P_i$  to  $P_{(i+1) \bmod N}$
- When process acquires token
  - Checks to see if it needs to enter critical section
  - If no, send token to neighbor
  - If yes, access resource
    - Hold token until done

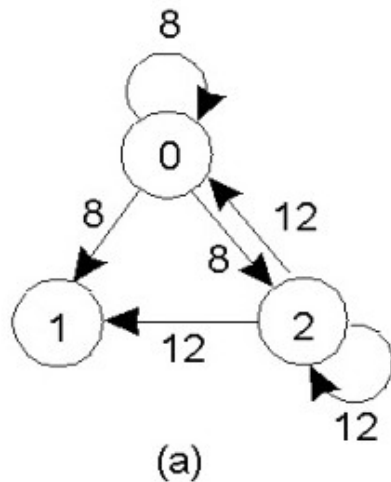


# Token Ring algorithm

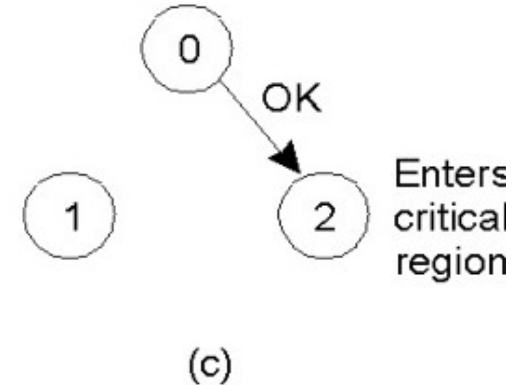
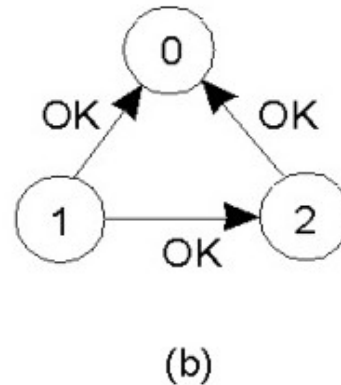
- Only one process at a time has token
  - Mutual exclusion guaranteed
- Order well-defined
  - Starvation cannot occur
- If token is lost (e.g. process died)
  - It will have to be regenerated
- Does not guarantee FIFO order

# Lecture 33

## A Distributed Algorithm



Enters  
critical  
region



- Two processes want to enter the same critical region at the same moment.
- Process 0 has the lowest timestamp, so it wins.
- When process 0 is done, it sends an OK also, so 2 can now enter the critical region.

# Lecture 34

## Comparison of these for Algorithm

# Read it by yourself

## Mutual Exclusion Algorithms Comparison

| Criteria                   | Centralized                         | Distributed                              | Decentralized                                  | Token Ring                             |
|----------------------------|-------------------------------------|------------------------------------------|------------------------------------------------|----------------------------------------|
| Control Mechanism          | Single coordinator (central server) | All nodes communicate to reach consensus | Multiple coordinators                          | Logical ring passes a token            |
| Request Message Complexity | $O(1)$ (request to coordinator)     | $O(n)$ (each node sends to all others)   | $O(1)$ to $O(n)$ , depending on load balancing | $O(1)$ (only token passing)            |
| Synchronization Delay      | Low                                 | Medium to high                           | Medium                                         | Low to medium                          |
| Bottleneck                 | Yes (coordinator)                   | No                                       | No                                             | No                                     |
| Single Point of Failure    | Yes                                 | No                                       | Reduced (due to redundancy)                    | Yes (token loss or node failure)       |
| Fairness                   | Depends on queueing at coordinator  | Generally fair (logical timestamps)      | Can be unfair depending on design              | Fair (token rotates through all nodes) |
| Fault Tolerance            | Low (if coordinator crashes)        | Higher (can tolerate failures)           | Medium to high (redundancy needed)             | Medium (token regeneration required)   |
| Message Overhead           | 2 messages per CS entry             | $2(n-1)$ messages per CS entry           | Variable (depends on architecture)             | 1 message per CS entry (token pass)    |